গভীর ডেক · D4

লাইফসাইকেল ও অপস

১২ মাইগ্রেশন · ২০ commit · ১টি পুরনো ট্যাগ · ব্যাকআপ ১৪ দিন · কোনো মনিটরিং নেই — সফটওয়্যার চালু হওয়ার পর কী হয়

মানচিত্র

তিনটি ব্লক: প্রস্তুতি → চালু → বাঁচিয়ে রাখা

কোড লেখা শেষ হওয়া মানে জীবন শুরু, শেষ নয়

১. প্রস্তুতি

স্লাইড ৩–৬ · রিহার্সাল স্ক্রিপ্ট · ১৩ দফা চেকলিস্ট · prod সেটিংস প্রমাণ

প্রমাণ: `check --deploy` → ০ সমস্যা

২. চালু

স্লাইড ৭–১০ · build → release → gunicorn · মাইগ্রেশনের ঝুঁকি · রোলব্যাক

প্রমাণ: Procfile ও build.sh

৩. বাঁচিয়ে রাখা

স্লাইড ১১–১৯ · ব্যাকআপ/রিস্টোর · মনিটরিং · লগ · ডেটা-জীবনচক্র · বিলিং রুটিন

প্রমাণ: `ops/` স্ক্রিপ্ট ও ৪টি row-count

এই ডেকের সবচেয়ে গুরুত্বপূর্ণ স্লাইডটি শেষের দিকের — **যা কখনো চালানো হয়নি** তার তালিকা। প্ল্যান আর প্রমাণের পার্থক্য সেখানেই।

শিক্ষকের নোট: শুরুতে বলুন: "অ্যাপ চালু করা একটি দিনের কাজ; চালু রাখা প্রতিদিনের কাজ।" শিক্ষার্থীদের অভিজ্ঞতা প্রায়ই প্রথমটিতে সীমিত — এই ডেক দ্বিতীয়টির কথা।

পথ

চারটি পরিবেশ, একটি কোড

প্রতিটির আলাদা ঝুঁকি, আলাদা দায়িত্ব

১. ডেমো

bash run.sh

SQLite · runserver

seed_demo ডেটা

কেউ ইমেইল পায় না

ঝুঁকি: শূন্য

২. রিহার্সাল

docs/deploy_check.sh

gunicorn :8010 · ২ worker

DEBUG=False · PostgreSQL

৫টি চেক + হেডার পরীক্ষা

ঝুঁকি: নিজের মেশিন

৩. প্রোডাকশন

build.sh → release → web

gunicorn · ৩ worker

WhiteNoise · আসল ডোমেইন

আসল স্কুল, আসল ডেটা

ঝুঁকি: সুনাম

৪. দৈনিক অপস

backup.sh · restore.sh (drill)

billing_report (cron 09:00)

লগ পড়া · ট্রায়াল শেষ হওয়া

প্রতিদিন ১০ মিনিট

ঝুঁকি: ভুলে যাওয়া

সৎ অবস্থান: ৩ নম্বর ধাপটি এখনো ঘটেনি — DEPLOYMENT.md নিজেই লিখেছে "not yet deployed to a public host"

অর্থাৎ ১, ২ ও ৪ (আংশিক) প্রমাণিত; ৩ কেবল প্রস্তুত — কোনো স্কুলের আসল ডেটা কখনো এই কোডে আসেনি

এই পার্থক্যটা লুকানো হয় না, কারণ লুকালে পরের মানুষ ভুল পরিকল্পনা করে

শিক্ষকের নোট: চতুর্থ বাক্সটি প্রথমে দেখান — শিক্ষার্থীরা ভাবে "ডিপ্লয়" মানে একটি কমান্ড। আসলে চতুর্থ ধাপটাই সবচেয়ে দীর্ঘ: প্রতি দিনের রুটিন।

রিহার্সাল

`deploy_check.sh` — ৩০ লাইনে প্রোডাকশন মহড়া

আসল gunicorn, আসল `prod` সেটিংস, তারপর ৫টি অনুরোধ

```
export DJANGO_SETTINGS_MODULE=config.settings.prod
python manage.py collectstatic --noinput
gunicorn config.wsgi:application \
    --bind 127.0.0.1:8010 --workers 2 --daemon
sleep 3

check "login page"         /accounts/login/      # 200
check "static css"         /static/css/ims.css   # 200
check "unknown page"       /does-not-exist       # 404
check "workspace anon"     /w/sunrise…/          # 302
```

যা যাচাই হয়	কেন এটিই
লগইন পাতা ২০০	অ্যাপ উঠেছে কি না
স্ট্যাটিক CSS ২০০	WhiteNoise ঠিক কাজ করছে
অচেনা URL → ৪০৪	নিজের এরর-পাতা, ডিফল্ট নয়
অ্যানোনিমাস workspace → ৩০২	টেন্যান্সি-রিডাইরেক্ট জীবিত
নিরাপত্তা-হেডার grep	prod.py-র পতাকাগুলো সত্যিই বেরোচ্ছে

ফাঁক: স্ক্রিপ্টটি `.env.production.local` ফাইল দাবি করে, যা রিপোতে নেই — তাই নতুন কেউ চালাতে গেলে নিজে বানাতে হয়। আর স্ক্রিপ্ট নিজে কোনো টেস্টে চলে না।

ধারণাটাই মূল: "ডিপ্লয়ের আগে ডিপ্লয়ের মহড়া"। `runserver`-এ কাজ করা মানে প্রোডাকশনে কাজ করা নয়।

শিক্ষকের নোট: জিজ্ঞেস করুন: "runserver আর gunicorn-এ কী পার্থক্য?" — runserver ডেভেলপমেন্টের জন্য, স্ট্যাটিক নিজে দেয়, DEBUG-নির্ভর আচরণ আছে; gunicorn প্রোডাকশনের, WhiteNoise ছাড়া স্ট্যাটিক নেই।

চেকলিস্ট

গো-লাইভের ১৩টি দফা

`DEPLOYMENT.md`-তে লেখা — প্রতিটির পেছনে একটি বাস্তব দুঃস্বপ্ন

দফা	না করলে
SECRET_KEY ৫০+ র‍্যান্ডম, সিক্রেট স্টোরে	সেশন/টোকেন জাল করা যায়
ALLOWED_HOSTS ও CSRF origins আসল ডোমেইন	prod.py শুরুই হবে না (ভালো)
HTTPS কাজ করে, http রিডাইরেক্ট	কুকি খোলা পথে যায়
"Forgot password" একবার পুরো চালানো	SMTP ভুল, ব্যবহারকারী আটকে
`check --deploy` → ০ সমস্যা	কোন সেটিং বাদ, অজানা
ব্যাকআপ চালেছে + রিস্টোর ড্রিল হয়েছে	ব্যাকআপ অস্তিত্ব প্রমাণহীন
ডেমো ডেটা মুছে, `DemoPass!2026` নেই	সবাই জানে পাসওয়ার্ড

দফা	না করলে
host-এ `runserver` নয়, gunicorn	ডেভেলপমেন্ট সার্ভার সর্বজনীন
অন্য কেউ শিক্ষক ও ছাত্র হয়ে ঢুকে বলেছে "শুধু নিজেরটাই দেখি"	অনুমতি কোডে ঠিক, চোখে দেখা হয়নি
`seed_plans` চালানো, দাম মিলছে	প্রাইসিং পাতায় মিথ্যা দাম
প্ল্যাটফর্ম অ্যাডমিন আসল মানুষের	`admin@ims.local` চিরকাল অ্যাডমিন
`media/` স্থায়ী ভলিউমে	রিডিপ্লয়ে লোগো মুছে যায়
`billing_report` দৈনিক	প্রথম ট্রায়াল শেষ হওয়া কেউ দেখে না

নবম দফাটি সবচেয়ে কম ব্যবহৃত, সবচেয়ে দরকারি: **অন্য একজন মানুষ ঢুকে বলবে "আমি শুধু নিজেরটাই দেখি"** — এটাই অনুমতির আসল পরীক্ষা।

শিক্ষকের নোট: চেকলিস্ট মুখস্থ করাবেন না — শিক্ষার্থীদের নিজের প্রজেক্টের জন্য ১০টি দফার তালিকা লিখতে বলুন। তালিকা লেখার অভ্যাসই আসল শেখা।

রিলিজ

একটি রিলিজের তিনটি ধাপ

build → release → web; তিনটির দায়িত্ব আলাদা

```
# build.sh
pip install -r requirements-prod.txt
python manage.py collectstatic --noinput
python manage.py migrate --noinput

# Procfile
release: python manage.py migrate --noinput
web: gunicorn config.wsgi:application \
     --bind 0.0.0.0:$PORT --workers 3 \
     --timeout 60 --access-logfile - \
     --error-logfile -
```

ধাপ	কেন আলাদা
build	নির্ভরতা + স্ট্যাটিক — একবার, ইমেজ তৈরি হওয়ার সময়
release	স্কিমা বদল — নতুন কোড চালু হওয়ার ঠিক আগে, একবার
web	অনুরোধ সামলানো — একাধিক worker, যেকোনো সময় বন্ধ হতে পারে

ঝুঁকি: ৩ worker একসাথে চালু হলে migrate একবারই হয় (`release` ধাপে) — কিন্তু worker মাঝপথে নতুন স্কিমা পেলে পুরনো কোড ভাঙতে পারে। ব্লু-গ্রিন ডিপ্লয় নেই।

--timeout 60: ৬০ সেকেন্ডে উত্তর না দিলে worker মারা যায় — PDF/রিপোর্ট বড় হলে এই সীমা ছুঁতে পারে।

শিক্ষকের নোট: "worker মাঝপথে নতুন স্কিমা" — এটি বোঝানো কঠিন কিন্তু জরুরি। উদাহরণ: কলামের নাম বদলালে পুরনো worker ভাঙবে; তাই দুই ধাপে নাম বদলানো হয় (যোগ → ব্যবহার → পুরনো মোছা)।

মাইগ্রেশন

১২টি মাইগ্রেশন — কেবল সামনের দিকে

৯ অ্যাপে `0001`, ৩ অ্যাপে `0002`; কোনো reverse migration লেখা নেই

ধাপ	কেন
১. নতুন কলাম যোগ, `null=True`	পুরনো সারি ভাঙে না
২. ডেটা মাইগ্রেশন দিয়ে পূরণ	কোড আর ডেটা আলাদা ফাইলে
৩. পরের রিলিজে `null=False`	তখন সব সারিতে মান আছে
৪. পুরনো কলাম মোছা (আরেক রিলিজ)	আগে কোড বদলাতে হয়

সৎ ফাঁক: `RunPython.noop` ছাড়া কোনো reverse লেখা নেই — অর্থাৎ ডেটা মাইগ্রেশন (`0002_seed_plans`) হুবহু ফিরিয়ে আনা যায় না।

```
# ডেটা মাইগ্রেশন (billing/0002)
def forwards(apps, schema_editor):
    Plan = apps.get_model("billing", "Plan")
    Subscription = apps.get_model(
        "billing", "Subscription")
    ...
    # পুরনো প্রতিটি workspace-এ
    # একটি Subscription সারি

operations = [migrations.RunPython(forwards)]
```

`apps.get_model()` — ভবিষ্যতের মডেল নয়, ওই মুহূর্তের মডেল। এটাই ডেটা মাইগ্রেশনের একমাত্র নিরাপদ নিয়ম।

মাইগ্রেশন = ডেটাবেজের লাইফসাইকেল: প্রতিটি ফাইল ইতিহাসের একটি পৃষ্ঠা। হাতে বদলাবেন না, ফাইল মুছবেন না — কেবল সামনে এগোবেন।

শিক্ষকের নোট: প্রশ্ন করুন: "মাইগ্রেশন ফাইল মুছে দিলে কী হয়?" উত্তর: অন্য সব পরিবেশের ইতিহাস আলাদা হয়ে যায় — আর কেউ আর জানে না কোনটা সত্য।

রোলব্যাক

ভাঙলে কী করবেন — রোলব্যাকের তিনটি পথ

আর একটি অপ্রিয় সত্য: ডেটা বদলালে রোলব্যাক সহজ নয়

পথ	কখন	কী খেয়াল
সামনে এগিয়ে ঠিক করা	বাগ ছোট, ডেটা অটুট — সবচেয়ে নিরাপদ	দ্রুত ঠিক করা + লগ দেখা; ইতিহাস সৎ থাকে
আগের commit-এ ফেরা	নতুন কোডই সমস্যা, স্কিমা বদলায়নি	`git revert` + রিডিপ্লয়; migrate ফেরাতে হয় না
ব্যাকআপ থেকে রিস্টোর	ডেটা নষ্ট হয়েছে (ভুল ডিলিট, ভুল আপডেট)	সব লেখা হারাবে — শেষ ব্যাকআপের পরের সব কাজ

এখানে সবচেয়ে সহজ পথ নেই: কোনো ভার্সন-ট্যাগ নেই (একটিমাত্র ট্যাগ `c073-slice1`, যা slice ১-এর)। অর্থাৎ "গত সপ্তাহের কোডে ফিরুন" — এই নির্দেশ দেওয়ার মতো কোনো চিহ্ন নেই।

২০টি commit: প্রতিটির বার্তা পড়ে বোঝা যায় কী বদলেছে — ভালো অভ্যাস। কিন্তু রিলিজ-চিহ্ন ছাড়া ২০টি commit মানে ২০টি সম্ভাব্য ফেরার বিন্দু, একটিও চিহ্নিত নয়।

সংশোধন (এক লাইনের কাজ): প্রতি রিলিজে `git tag -a v1.4 -m "…"` — তখন "গত ট্যাগে ফেরো" একটি সম্পূর্ণ নির্দেশ হয়ে যায়।

শিক্ষকের নোট: শিক্ষার্থীদের জিজ্ঞেস করুন: "ডেটা বদলে গেলে রোলব্যাক করা যায়?" উত্তর: কোড ফেরানো যায়, ডেটা নয় — কেবল ব্যাকআপ থেকে, আর তাতে নতুন কাজ হারায়। তাই ডেটা-বদলানো রিলিজ আলাদা করে, ছোট করে।

নজর

মনিটরিং নেই — কিন্তু কী দেখতে হতো

পাঁচটি সংকেত, যার একটিও আজ দেখা যায় না

সংকেত	কী বলত	আজ কী আছে
জীবিত আছে?	প্রতি ৩০ সেকেন্ডে ২০০	`/healthz` নেই — গেটে অনুমতি আছে, ঠিকানা নেই
এরর হার	৫০০-এর সংখ্যা হঠাৎ বাড়লে সতর্কতা	Sentry নেই; `gunicorn` error log কেউ পড়ে না
ধীরতা	p95 রেসপন্স সময়	কোনো মাপ নেই — "ধীর লাগছে" কেবল অনুভব
ডেটাবেজ	সংযোগ সংখ্যা, বড় কোয়েরি	`CONN_MAX_AGE=60` সেট, কিন্তু নজর নেই
ব্যবসায়িক	দৈনিক কত স্কুল সক্রিয়	`billing_report` হাতে চালাতে হয়

পরিণতি: সমস্যা জানা যায় ব্যবহারকারীর ফোনে। ডেমো-প্রজেক্টে গ্রহণযোগ্য; আসল স্কুলে নয় — কারণ প্রথম জানার অধিকার গ্রাহকের নয়, অপারেটরের।

শিক্ষকের নোট: প্রশ্ন করুন: "গ্রাহক আগে জানবে, না আপনি?" — এই একটি প্রশ্নই মনিটরিংয়ের পুরো যুক্তি। প্রথম সংকেতটি (healthz) এক ঘণ্টার কাজ।

লগ

লগ কোথায় যায়, আর কী কখনো যাবে না

সব লগ `stdout`-এ — ফাইলে নয়, তাই ঘুরিয়ে ফেলা সহজ

```
LOGGING = {
  "handlers": {"console": {"class":
      "logging.StreamHandler"}},
  "root": {"handlers": ["console"],
           "level": env("LOG_LEVEL", "INFO")},
  "loggers": {
    "django.request": {"level": "WARNING"},
    "django.security": {"level": "WARNING"},
  },
}
```

gunicorn-এর `--access-logfile -` আর `--error-logfile -` — দুটোই একই স্রোতে।

নিয়ম	কারণ
ফাইলে লেখা হয় না	host/stdout সংগ্রহ করে; অ্যাপ নিজে ফাইল ঘোরাতে হয় না
`django.request` WARNING	প্রতিটি 4xx/5xx একটি এন্ট্রি — কিন্তু কেউ পড়ে না
মেয়াদ host-এর হাতে	অ্যাপের জানা নেই লগ কত দিন থাকে

অডিট মডেলের নিয়ম: "পাসওয়ার্ড, রিসেট টোকেন বা সিক্রেট কখনো সংরক্ষণ করবেন না" — `AuditLog`-এ কেবল শনাক্তকারী আর সংক্ষিপ্ত সারাংশ।

তিন ধরনের লগ, তিন জায়গা: অ্যাপ-এরর → stdout · ব্যবসায়িক কাজ → `AuditLog` টেবিল · অ্যাক্সেস → gunicorn। কোনটা কোথায়, তা জানা থাকলে সমস্যার সময় খোঁজা সহজ।

শিক্ষকের নোট: প্রশ্ন করুন: "ব্যবহারকারী বলল 'আমার নম্বর বদলে গেছে' — কোথায় খুঁজবেন?" উত্তর: `AuditLog` (কে বদলাল), তারপর gunicorn access log (কখন), শেষে অ্যাপ লগ (এরর ছিল কি না)।

ব্যাকআপ

`backup.sh` — ৬টি কার্যকর লাইন

`pg_dump -Fc` · media tar · ১৪ দিন · `set -euo pipefail`

```
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
DEST="${BACKUP_DIR:-./backups}"
mkdir -p "$DEST"
pg_dump -Fc -f "$DEST/ims-$STAMP.dump"
tar -czf "$DEST/media-$STAMP.tar.gz" media/ || true
find "$DEST" -name '*.dump' -mtime +14 -delete
```

সিদ্ধান্ত	কারণ
ডেটাবেজ + media আলাদা	লোগো/ম্যাটেরিয়াল ডাম্পে থাকে না
`-Fc` ফরম্যাট	টেবিল ধরে ধরে ফেরানো যায়
১৪ দিন	দুই সপ্তাহের মধ্যে ভুল ধরা পড়ে
`set -euo pipefail`	মাঝপথে ব্যর্থ = ব্যাকআপ নেই, আধা-ব্যাকআপ নয়

সীমা: ডিফল্ট গন্তব্য `./backups` — অর্থাৎ সার্ভারের ভেতরেই। সার্ভার হারালে ব্যাকআপও যায়। অফসাইট কপি নেই, এনক্রিপশন নেই, WAL/PITR নেই।

আরও একটি: `pg_dump` চালানোর অনুমতি ও ডাটাবেজ-নাম কোথাও লেখা নেই — স্ক্রিপ্ট ধরে নেয় আপনি ওই মেশিনে PostgreSQL-এর সুপারইউজার। বাস্তবে এটি প্রথমেই আটকে যায়।

শিক্ষকের নোট: "ব্যাকআপ আছে" বাক্যটি কখনো প্রমাণ নয়। যা প্রমাণ: একটি ফাইল, একটি তারিখ, একটি আকার — আর একটি সফল রিস্টোর।

ড্রিল

`restore.sh` — ব্যাকআপ ফেরানোর নিরাপদ পথ

আসল ডেটাবেজে কখনো সরাসরি রিস্টোর নয়

```
dropdb --if-exists "$SCRATCH"
createdb "$SCRATCH"
pg_restore -d "$SCRATCH" "$DUMP"

psql -d "$SCRATCH" -c "select 'students', count(*)
   from students_studentprofile
 union all select 'attendance', count(*)
   from attendance_attendancerecord
 union all select 'marks', count(*)
   from assessments_mark
 union all select 'audit', count(*)
   from audit_auditlog;"
```

নীতি	কারণ
স্ক্র্যাচ ডেটাবেজে প্রথমে	ভাঙা ডাম্প চালালে আসল ডেটা যায় না
৪টি টেবিলের সংখ্যা ছাপা	"ফেরানো হয়েছে" নয়, "কত সারি ফিরল" — যাচাইযোগ্য
swap কমান্ড মানুষ চালাবে	শেষ সিদ্ধান্ত স্বয়ংক্রিয় নয় — কারণ সেটি অপূরণীয়

সৎ ফাঁক: ড্রিলের ফল কোথাও লিপিবদ্ধ হয় না — কে, কবে, কত সারি ফিরে পেল, তার কোনো রেকর্ড নেই। মাসিক ড্রিল এক লাইনের কাজ, তবু হয় না।

মূলমন্ত্র: যে ব্যাকআপ কখনো ফেরানো হয়নি, সেটি ব্যাকআপ কি না জানা নেই — এমনকি ফাইলটি থাকলেও।

শিক্ষকের নোট: শিক্ষার্থীদের ছোট কাজ দিন: নিজের ডেমো-ডেটাবেজের একটি ডাম্প নিয়ে স্ক্র্যাচে ফিরিয়ে ৪টি সংখ্যা মেলাতে। যারা একবার করেছে, তারাই আসলে জানে।

জীবনচক্র

স্কুল চালু থেকে বন্ধ — চারটি অবস্থা

কোনো অবস্থাতেই ডেটা মোছা হয় না

অবস্থা	স্কুল যা দেখে	কী চলে	কী বন্ধ
ট্রায়াল	নীল ব্যানার + দিন-গণনা	সব	—
মেয়াদোত্তীর্ণ / past due	"Read-only mode"	পড়া + এক্সপোর্ট	সব লেখা
স্থগিত (প্ল্যাটফর্ম থেকে)	"Account suspended" + কারণ	পড়া	সব লেখা
বাতিল	পড়া	এক্সপোর্ট	সব লেখা

কারিগরি রূপ: `SubscriptionGateMiddleware` কেবল `SAFE_METHODS` ছাড়া বাকি সব থামায়; `ALLOW_PREFIXES` লগইন, রিনিউ, প্ল্যাটফর্ম ও স্ট্যাটিক খোলা রাখে।

যা এখনো নেই: UI-তে "স্কুল মুছুন" বাটন নেই। কেউ চাইলে `backup.sh` চালিয়ে Django admin থেকে workspace মুছতে হয় — অর্থাৎ মুছে ফেলা একটি **হাতে করা** অপারেশন, নকশা নয়।

নীতি: **অর্থ না দিলে ডেটা জিম্মি হয় না।** স্কুল তার রেকর্ড এক্সপোর্ট করে নিয়ে যেতে পারে — এটি চুক্তির নয়, কোডের নিয়ম।

শিক্ষকের নোট: এই স্লাইডটি নৈতিক সিদ্ধান্তের — "না দেওয়া মানে ডেটা আটকে রাখা নয়"। শিক্ষার্থীদের জিজ্ঞেস করুন: "এক স্কুল যদি সিস্টেম বদলাতে চায়, তার ডেটা নেওয়ার অধিকার আছে কি?"

ডেটা

কী জমা থাকে, কত দিন, আর কে দেখতে পারে

শিশুর ডেটা রাখা মানে বাড়তি দায়িত্ব

ডেটা	কোথায়	নিয়ম
ছাত্রের নাম, অভিভাবকের ফোন, জন্মতারিখ	`StudentProfile`	স্কুল নিজের ডেটা; অপারেটর কাজে ব্যবহার করে না
হাজিরা ও নম্বর	`AttendanceRecord` · `Mark`	কখনো মুছে ফেলা হয় না — সংশোধন হয়, লিপিবদ্ধভাবে
অডিট লগ	`AuditLog`	কে বদলাল, কখন, কেন — ৩০০ অক্ষরে
ইমেইল ও পাসওয়ার্ড	`User`	পাসওয়ার্ড হ্যাশ; কখনো লগে নয়, কখনো অডিটে নয়
ব্যাকআপ	সার্ভারের ডিস্ক	**১৪ দিন** — তারপর স্বয়ংক্রিয় মুছে যায়

সৎ ফাঁক: কোনো গোপনীয়তা-নীতি (privacy policy) বা ডেটা-মুছে-ফেলার নীতি লেখা নেই — `LAUNCH_PLAYBOOK` নিজেই বলেছে "Terms + privacy page — not in this repo yet"।

ব্যবহারিক নিয়ম: ব্যাকআপ ফাইলে পুরো স্কুলের ডেটা থাকে — সেটি হাতে পাওয়া মানে সব ডেটা হাতে পাওয়া। তাই ব্যাকআপ ডিস্কের অনুমতিও সমান গুরুত্বপূর্ণ।

শিক্ষকের নোট: "শিশুর ডেটা" — এই শব্দ দুটি জোর দিন। প্রাপ্তবয়স্ক ব্যবহারকারীর ডেটার চেয়ে ভিন্ন দায়িত্ব, ভিন্ন আইন, ভিন্ন নৈতিকতার প্রশ্ন।

রুটিন

প্রতিদিনের ১০ মিনিট — চারটি কাজ

অপস মানে বীরত্ব নয়, অভ্যাস

```
0 9 * * * cd /srv/ims && .venv/bin/python \
  manage.py billing_report | mail -s "IMS" you@x.com

0 2 * * * cd /srv/ims && BACKUP_DIR=/var/backups/ims \
  bash ops/backup.sh
```

প্রথম লাইনটি `LAUNCH_PLAYBOOK.md`-তে লেখা আছে; দ্বিতীয়টি লেখা নেই — কিন্তু দরকার।

কাজ	কী দেখবেন
`billing_report`	সক্রিয় স্কুল, MRR, ৭ দিনে শেষ হওয়া ট্রায়াল, unpaid ইনভয়েস
ব্যাকআপের ফাইল	আজকের ডাম্প আছে? আকার গতকালের মতো?
এরর-লগ	গত ২৪ ঘণ্টায় নতুন ৫০০?
ট্রায়াল-শেষ রিপোর্ট	স্কুলকে মনে করানো — রিনিউয়ের আসল কাজ

প্রথম ট্রায়াল-শেষ হাতে করতে হবে: স্বয়ংক্রিয় রিমাইন্ডার নেই — ১৪ দিন পর স্কুল চুপচাপ read-only হয়ে যাবে, আর কেউ টের পাবে না। `LAUNCH_PLAYBOOK` এটাই স্বীকার করেছে।

শিক্ষকের নোট: ক্রন-লাইনটি বোর্ডে লিখুন। "প্রতিদিন একটি ইমেইল" — এটাই সবচেয়ে সস্তা মনিটরিং, যা আজও কাজ করছে।

সংকট

কিছু ভাঙলে — পাঁচটি ধাপ

ক্রমটাই গুরুত্বপূর্ণ; প্রথম ধাপ কখনো "ঠিক করা" নয়

#	ধাপ	IMS-এ বাস্তবে
১	**থামাও**	ভুল মাইগ্রেশন চললে ডিপ্লয় থামান; সাইট নামানো দরকার হলে নামান
২	**মাপো**	কত স্কুল, কত সারি, কোন সময় থেকে — `AuditLog` + gunicorn log
৩	**আগে ব্যাকআপ**	**যেকোনো সংশোধনের আগে** `ops/backup.sh` — নাহলে ফেরার পথ নেই
৪	**ছোট করে ঠিক করো**	একটি commit, একটি কারণ; বড় রিফ্যাক্টর সংকটের সময় নয়
৫	**লিখে রাখো**	কী ঘটল, কেন, কী বদলাল — নাহলে দ্বিতীয়বার একই ভুল

উদাহরণ (কাল্পনিক নয়): কোনো স্কুলের সাবস্ক্রিপশন ভুলে মেয়াদোত্তীর্ণ দেখালে — গেট সব লেখা বন্ধ করবে, কিন্তু ডেটা অটুট। সমাধান বিলিং ডেটা ঠিক করা, রিস্টোর নয়।

সবচেয়ে খারাপ প্রতিক্রিয়া: আতঙ্কে সেটিং ঢিলা করা (`DEBUG=True` , গেট বন্ধ)। এটি সমস্যা থামায় না, লুকায়।

শিক্ষকের নোট: ৩ নম্বর ধাপে থামুন: "সংশোধনের আগে ব্যাকআপ"। শিক্ষার্থীরা উল্টো করে — আগে ঠিক করে, তারপর ভাবে। ভুল ডেটা বদলালে ফেরার পথ আর থাকে না।

ধারণক্ষমতা

একটি স্কুল এক বছরে কত সারি বানায়

হিসাবটা সরল; মাপাটা হয়নি — দুটোই সত্য

হিসাব (৮০০ ছাত্রের স্কুল)	সারি/বছর
হাজিরা: ৮০০ × ৮ বিষয় × ২০০ দিন	১২,৮০,০০০
নম্বর: ৮০০ × ৮ × ৩ পরীক্ষা	১৯,২০০
অডিট: প্রতিদিন ~২০০ কাজ	৭৩,০০০
এনরোলমেন্ট + হোমওয়ার্ক + সাবমিশন	~৩০,০০০

মূল চাপ হাজিরায় — একাই বাকি সব মিলিয়ে যা হয় তার প্রায় ১০ গুণ।

সম্পদ	আজকের অবস্থা
gunicorn worker	৩টি · timeout ৬০ সে
DB সংযোগ	`CONN_MAX_AGE=60` · worker-প্রতি একটি
ডিস্ক	`media/` স্থায়ী ভলিউমে থাকতে হবে
লোড-টেস্ট	**কখনো হয়নি**

তাই "১০০০ ব্যবহারকারী চলবে?" প্রশ্নের সৎ উত্তর: "জানি না — মাপা হয়নি।" ইনডেক্স আছে, কিন্তু `EXPLAIN` করে দেখা হয়নি।

পরের ধাপ স্পষ্ট: বড় স্কুলের ডেমো-ডেটা বানিয়ে (৫০ হাজার সারি) আগে `EXPLAIN ANALYZE`, তারপর দরকার হলে লোড-টেস্ট। অনুমান দিয়ে স্কেলিং নয়।

শিক্ষকের নোট: হিসাবটি বোর্ডে করুন — শিক্ষার্থীরা অবাক হয় দেখে যে হাজিরাই ৯০% ডেটা। এখান থেকেই বোঝা যায় কেন `(class_group, date)` ইনডেক্স সবচেয়ে জরুরি।

সততা

যা লেখা আছে, কিন্তু কখনো চালানো হয়নি

প্ল্যান আর প্রমাণের পার্থক্য — এই স্লাইডটাই সেই পার্থক্য

জিনিস	অবস্থা	প্রথমবার চালালে যা ভাঙতে পারে
`ops/backup.sh`	লেখা, চালানো হয়নি	`pg_dump`-এর অনুমতি, `media/` না থাকলে ব্যর্থ
`ops/restore.sh`	লেখা, ড্রিল হয়নি	স্ক্র্যাচ DB-র অনুমতি; swap ধাপ হাতে
`docs/deploy_check.sh`	লেখা, দরকার `.env.production.local`	ফাইলটি রিপোতে নেই — নতুনদের প্রথম বাধা
প্রোডাকশন ডিপ্লয়	**কখনো হয়নি**	DEPLOYMENT.md নিজেই স্বীকার করেছে
লোড-টেস্ট / EXPLAIN	হয়নি	ধীরতার প্রশ্নের উত্তর নেই
ট্যাগ-ভিত্তিক রিলিজ	শুধু একটি পুরনো ট্যাগ	"গত রিলিজে ফেরো" বলা যায় না

এগুলো ব্যর্থতা নয় — **অসমাপ্ত কাজ**। পার্থক্য একটাই: অসমাপ্ত কাজের তালিকা থাকে, ব্যর্থতা লুকানো থাকে।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টে এমন একটি তালিকা লিখতে — "যা লিখেছি কিন্তু চালাইনি"। বেশিরভাগের তালিকায় ডিপ্লয়, ব্যাকআপ আর টেস্ট থাকবে।

সারমর্ম

ছয়টি নীতি, আর চারটি কাজ

অপস শেখা যায় তিনভাবে: পড়ে, করে, আর ভুল করে

- **যে ব্যাকআপ ফেরানো হয়নি, সেটি ব্যাকআপ নয়** — ফাইল থাকলেই প্রমাণ হয় না
- **মাইগ্রেশন কেবল সামনে** — ডেটা বদলালে রোলব্যাক মানে ব্যাকআপ, কোড নয়
- **সংশোধনের আগে ব্যাকআপ** — উল্টো ক্রমে করলে ফেরার পথ হারায়
- **গ্রাহকের আগে আপনার জানা উচিত** — মনিটরিং এজন্যই, সৌন্দর্য নয়
- **অর্থ না দিলে ডেটা জিম্মি হয় না** — read-only, কখনো মুছে ফেলা নয়
- **মাপা হয়নি মানে জানা নেই** — ধারণক্ষমতার প্রশ্নে "জানি না" বলাই সৎ উত্তর

চারটি কাজ (আজ): ১) প্রতি রিলিজে tag · ২) `/healthz` · ৩) ক্রনে backup + মাসিক restore drill · ৪) `.env.example`-এ `DB_*` যোগ করা।

শিক্ষকের নোট: চারটি কাজের প্রতিটি ১০–৩০ মিনিটের। শিক্ষার্থীদের একটি বেছে আজই করতে বলুন — শেখার সবচেয়ে কার্যকর রূপ।

D5 — ডিজাইন সিস্টেম

সিস্টেম চলছে; এখন তাকে দেখতে যেমন লাগে, তেমন বানানো · ১৮ স্লাইড